PATCHWORK: Frontend Spec

Brief for building the PATCHWORK dashboard UI. Covers what each screen must show and do, the API it calls, and the rules that must not break.

---

1. What PATCHWORK is (30-second version)

A site adds one `<script>` tag. That script fetches **patches** from PATCHWORK and applies them in the visitor's browser: change text, change style, hide an element, or set/remove an attribute. The site's code never changes. Remove a patch and the original site is back.

Think of the site as a printed page. A patch is a sticky note on top of it.

Demo story (what the UI must make look great):

1. The Northfold store's Checkout button is broken: blank orange block, clicking it does nothing.
2. In PATCHWORK's editor: click Checkout, fix its colour and attribute, see it fixed live, save, preview, publish v1.
3. The real store is fixed in a normal tab. No code changed.
4. Versions page: roll back to v0 (broken again), forward to v1 (fixed). Show the audit log.
5. "Patch is live. Want me to open a PR?" Click Yes, see the diff, open a real GitHub PR.

Full story is ~3 minutes on stage. The UI must be readable from the back of the room, so use big type, clear states and no clutter.

---

2. Stack and ground rules

- **Next.js App Router + TypeScript + Tailwind.** Already set up in the repo (`nexus-patchwork`).

- Pages live in `app/`. Server components read data directly via `db` from `@/lib/supabase` (server-only). Client components call the API routes with `fetch`.
- **Never import `@/lib/supabase` or `@/lib/workingSet` in a `"use client"` file.** The service role key must never reach the browser.
- **Do not touch** `public/snippet.js` or `public/editor.js`. They run on the customer's site and are fail-safe critical.
- **Single site, no login.** Every page uses the first (only) row in `sites`.
- Run: `npm run dev` (dashboard on http://localhost:3000). Store: `npx serve -l 5500` in the `northfold_store` repo.
- There is a `.input` CSS class in `app/globals.css` used for text inputs. Reuse it or replace it consistently.
- `app/layout.tsx` still has the default metadata ("Create Next App"). Change the title to "PATCHWORK".

---

3. Screens

Four pages, plus a shared top nav (PATCHWORK logo/name, then Patches · Editor · Versions · Log). The current nav is plain underlined links, so it needs real styling.

3.1 HOME / PATCHES: `APP/PAGE.TSX` (CURRENTLY A STUB WITH 3 LINKS)

The main dashboard. Must show:

- **Site card:** site name (Northfold), URL, which version is live (e.g. "Live: v1"), and the snippet tag to copy: `<script src="http://localhost:3000/snippet.js" data-key="<site_key>"></script>` with a Copy button.

- **Patch list:** one row per patch:
  - Name (e.g. “Fix Checkout button”)
  - Status badge: `draft` (grey/yellow), `published` (green), `retired` (muted/struck through)
  - **Broken warning** if `last_broken_at` is set: e.g. a red “⚠ Couldn’t find element on a visitor’s page, 3 min ago”. This is a warning only. It does not change status.
  - Summary of changes, e.g. `style: color` · `attribute: data-checkout`
  - Target hint: `target.selector` in monospace (truncate)
  - `pr_url` link if a PR has been opened (“PR opened ↗”)
  - Updated time
  - Actions:
    - **Edit**: open the editor for this patch (nice-to-have; see §5)
    - **Make it permanent**: opens the Patch-to-PR flow (§3.5)
    - **Retire**: sets status `retired`; patch drops out of the next published version
- **Empty state:** “No patches yet. Open the editor to create one.” with a button to `/editor`.
- Primary CTA: **Open editor**.

3.2 EDITOR: `app/editor/page.tsx` + `app/editor/editor.tsx` (WORKING, NEEDS VISUAL POLISH)

This is the star screen. The logic works. **Restyle it, do not rewrite the message logic.**

Layout: store loaded in an `<iframe>` on the left (big), side panel on the right (~320px).

Top bar:

- Title + site name
- **Browse / Select toggle** (segmented control). Must be very obvious which mode is on.
  - **Browse:** the store works normally (needed to add an item and open the bag drawer so the Checkout button exists).
  - **Select:** hovering outlines elements; clicking selects one (the click is blocked from the store).
- Links to Versions / Home

Side panel, **nothing selected:** instructions: “Use **Browse** to get to the element (e.g. open the bag), then switch to **Select** and click it.”

Side panel, **element selected:**

- Selected element’s selector in monospace (read-only)
- **Patch name** (prefilled, e.g. `Fix "Checkout"`)
- **Text**: disabled if the element has nested children, with a hint explaining why
- **Text colour**: free-text input (must accept `var(--on-accent)`, a CSS variable, as well as hex) plus a colour picker. Placeholder shows the element’s current computed colour.
- **Background**: same as text colour
- **Font size**: placeholder = current value
- **Hide element** checkbox
- **Attribute**: name + value inputs
- **Save as draft** / **Update draft** button: disabled while saving, or when there are no changes, or when the name is empty
- Status line: “Saving…”, “Saved as draft”, “Draft updated”, or “Save failed: ”

Every form change is applied live in the iframe instantly. That is the “wow” moment, so keep it snappy.

Nice additions (only if time allows):

- After save, show **Preview** and **Go to Publish** buttons right in the panel so the demo flows without hunting.

3.3 VERSIONS: `APP/VERSIONS/PAGE.TSX` + `ACTIONS.TSX` (WORKING, NEEDS POLISH)

- **Working set card** (top):
  - ”N patch(es) would go live, including M draft(s): …”
  - **Note** input (optional) + **Publish** button
  - **Open preview** link: opens the store with `?pw_preview=<preview_token>` in a new tab. Shows drafts to that viewer only.

- **After publish succeeds:** show the prompt **"Patch is live. Want me to open a PR to fix this in the code?"** with **Yes** / **Not now**. Yes opens the Patch-to-PR flow (§3.5). *(Not built yet.)*
- **Version list**, newest first:
  - `v4` big and monospace
  - Note (or patch names, or "No patches")
  - Date + patch count
  - **Live** badge (green) on the live one
  - **Make vN live** button on all others (works for both rollback and roll-forward)
- Make the live version unmistakable. On stage the judge must see v0 → v1 flip instantly.

3.4 AUDIT LOG: `APP/LOG/PAGE.TSX` (WORKING, NEEDS POLISH)

Newest-first list. Each row: time, action, description.

action	description shown
`created`	patch name
`edited`	patch name
`published`	`v1 (Fix Checkout button, …)`
`rolled_back`	`v1 → v0`
`pr_opened`	patch name (+ PR link from `details`, once built)
`reported_broken`	patch name
`retired`	patch name

Give each action its own colour/icon so the log scans fast.

3.5 PATCH TO PR FLOW (MODAL OR PAGE; NOT BUILT YET, BACKEND IN PROGRESS)

Entry points: "Yes" on the post-publish prompt, or "Make it permanent" on a patch row.

States:

1. **Working…**: “Finding the right files in `keithdmonte/northfold_store`…” (can take 5-20s; Claude is writing the fix). Show a spinner and keep the user informed.
2. **Diff review:** per file (e.g. `styles.css`, `app.js`), show a before/after diff of each `find` → `replace` edit. Red/green lines. Monospace.
3. Buttons: **Open PR** / **Cancel**.
4. **Opening PR…**
5. **Done:** “PR opened” + link to the GitHub PR. The patch row now shows the PR link.
6. **Error:** clear message + Retry. (E.g. “Couldn’t find a safe edit for this patch.”)

If time is short, the diff can be plain text (this is first on the cut list).

---

4. API contract

All JSON. Errors return `{ "error": "message" }` with a 4xx/5xx status. Always show `error` to the user.

EXISTS NOW

`POST /api/patches`: create draft

```
// request
{ "site_id": "uuid", "name": "Fix Checkout button", "page": "*",
"target": { ... }, "changes": [ ... ] }
// response
{ "patch": { "id": "uuid", "name": "...", "status": "draft", ... } }
```

`PATCH /api/patches`: update an existing patch

```
{ "id": "uuid", "name": "...", "target": { ... }, "changes": [ ... ] }
// -> { "patch": { ... } }
```

`POST /api/publish`: freeze the working set into a new version and make it live

```
{ "site_id": "uuid", "note": "Fix checkout" }
// -> { "version": { "id": "uuid", "number": 1 } }
```

`POST /api/rollback`: make any version live

```
{ "version_id": "uuid" }
// -> { "ok": true }
```

NOT BUILT YET (BACKEND SIDE; COORDINATE WITH KEITH)

- `GET /api/patches`: list patches (or read via `db` in a server component on the home page, as the other pages do)
- Retire a patch (likely `PATCH` / `DELETE /api/patches` setting `status: "retired"`)
- `POST /api/pr`: two steps: propose (returns edits/diff) and confirm (opens the PR, returns `pr_url`). Exact shape TBD. Build the UI against a mock first.

---

5. Data shapes

sites: `id, name, url, site_key, preview_token, github_repo, live_version_id, created_at`

patches

```
{
  id: string; site_id: string;
  name: string;              // "Fix Checkout button"
  page: string;              // "*" = all pages
  target: {                  // clues used to find the element
    tag: string; text: string | null; classes: string[];
    attrs: Record<string, string>;
    parent: { tag: string; classes: string[] } | null;
    selector: string;
  };
  changes: Change[];
  status: "draft" | "published" | "retired";
  last_broken_at: string | null;   // warning flag only
  pr_url: string | null;
  created_at: string; updated_at: string;
}

type Change =
  | { type: "text"; value: string }
  | { type: "style"; props: Record<string, string> }
  | { type: "hide" }
  | { type: "attribute"; name: string; value: string | null }; // null =
remove
```

versions: `id, site_id, number, patches (frozen copy, array), note, created_at`. `v0` = original site, no patches.

audit_log: `id, site_id, action, patch_id, version_id, details (json), created_at`

Working set = every patch whose status is not `retired`. Preview shows exactly this; Publish freezes exactly this.

---

6. Rules that must not break

1. **Do not change the `postMessage` logic in `Editor.tsx`.** Messages: `pw:ready`, `pw:select` (from iframe); `pw:mode`, `pw:apply` (to iframe). The `onMessage` handler must keep checking `e.origin === siteOrigin` **and** `e.source === iframe.contentWindow`. Restyling JSX around it is fine.

2. **Keep the iframe `src`** (`site.url` + `?pw_edit=<preview_token>`). Without it, the editor script doesn't load.
3. **No Supabase in client components.** Server components or API routes only.
4. **Text inputs stay plain strings.** Never render patch text/values with `dangerouslySetInnerHTML`.
5. Colour inputs must stay **free text** (CSS variables like `var(--on-accent)` are the actual demo fix).
6. Don't add features beyond this doc without checking with Keith (scope is tight: ~10h hackathon).

---

7. Design direction

- Clean, developer-tool feel (think Vercel / Linear). Light theme is fine; dark optional.
- High contrast, generous size: this is shown on a projector.
- Status colours used consistently everywhere: draft = amber, published/live = green, retired = grey, broken = red.
- Monospace for selectors, versions (`v1`), diffs.
- Every async button shows a busy state ("Publishing…", "Switching…", "Saving…") and an inline error on failure.
- Fast, no page reloads where avoidable (current pages use `router.refresh()` after actions, which is fine).

---

8. Priority order

1. Editor polish (§3.2): the demo's centrepiece
2. Versions page + post-publish PR prompt (§3.3)
3. Home / Patches list with broken warning, retire, "Make it permanent" (§3.1)
4. Patch-to-PR modal (§3.5), mocked until `/api/pr` exists
5. Audit log polish (§3.4)
6. Shared nav, page title, empty/loading states

9. Definition of done

Run the demo (§1) end to end in the browser: select Checkout in the editor, fix it live, save, preview, publish, roll back, roll forward, view the log, and walk through the PR flow. No console errors, every button shows feedback, and everything is readable on a projector.